School of Computer Science

COMP20280: Data Structures
Semester II

Assignment I
On: 24/03/2025
Due: See details in Lectures, poster session 23/04/2025

In this assignment, you will implement a randomised binary search tree known as a Treap (Tree + Heap) (Seidel and Aragon, 1996). You will analyse its performance and compare against your own AVLTreeMap and Java's built-in TreeMap. You will also benchmark the sorting capabilities of your implementation against other sorting algorithms.

The goal of this assignment is to:

- Understand and implement randomized balanced binary search trees
- Analyse algorithmic complexity of data structure operations
- Conduct empirical performance testing and benchmarking
- Compare different implementations on common sorting applications

You will work in the same teams you were assigned in the "Algorithms" module (if you have any questions about the teams, let me know).

The submission will be graded on the code, a short report and a poster.

The individual components are:

- code implementations, commited to github
- complexity and performance analyses
- unit tests

The team components are:

- Joint report (2 pages) detailing the complexity and performance analyses, eg. tables, charts and discussion. The report should include these sections:
  - Brief explanation of how a Treap data structure works
  - Design decisions in your own implementation of the Treap
  - Analysis of the performance results
  - Analysis of the sorting algorithms
  - When would you prefer a Treap over an AVLTreeMap?
- Poster for the session on 23rd April.

You will also prepare a poster which will be displayed and assessed on April 23rd between 09:30-13:00.

Q1: _____ **(40points)**

Implement the Treap in Java.

Follow your implementation of the TreeMap in the lectures and base your code on your existing work. Your Treap should be a fully generic data structure. The ADT for the Treap is the same as that of the TreeMap. A Treap is a self-balancing binary search tree that uses random priorities to maintain balance probabilistically. Each node in a Treap has:

- A key (which follows BST property)
- A priority (random value that follows heap property)

provide a full ADT?

Include unit tests for each method in your Treap.

Push your implementation to your GitHub repo.

Q2: _____ **(30points)**

Performance Comparison

Measure the performance of your Treap implementation against your `AVLTreeMap` implementation and also against Java's `java.util.TreeMap`.

Measure the time taken for the following operations on all 3 data structures [`Treap`, `AVLTreeMap`, `java.util.TreeMap`]

- Insertion (single and batch)
- Search (successful and unsuccessful)
- Deletion
- In-order traversal (iteration over all elements)

Test with different input sizes and patterns:

- range of sizes $n = [100, \ldots, 10000]$
- Random data
- Sorted data (ascending)
- Sorted data (descending)
- Partially sorted data

A table or chart detailed your performance measurements is required.

Q3: _____ **(30points)**

Sorting

Implement a sorting scheme using your `Treap` data structure and benchmark it against other sorting algorithms:

- `TreapSort`: Insert all elements into a `Treap` and retrieve them with in-order traversal
- `PQSort` Insert all elements into a priority queue (use your own priority queue, or `java.util.PriorityQueue`)
- Java's Collections.sort() (which uses TimSort)
- Quick Sort (using your implementation from Algorithms Module)
- Merge Sort (using your implementation from Algorithms Module)

Your benchmark should:

- Measured execution time for each algorithm
- Test with various input sizes $n = [100, \ldots, 10000]$
- Test with different input patterns (random, nearly sorted, reverse sorted)
- Report on memory usage if possible

References

Seidel, Raimund and Cecilia R Aragon (1996). "Randomized search trees". In: Algorithmica 16.4, pp. 464–497. URL: https://faculty.washington.edu/aragon/pubs/rst89.pdf.